Q. 1 (A) Explain double link list with its advantage and write a algorithm to insert node at beginning.

D**oubly Linked List (DLL)** is a type of linked list where **each node has three parts**:

1. **Data** – stores the value
2. **Prev pointer** – points to the previous node
3. **Next pointer** – points to the next node

Because every node is connected in **both directions**, you can move **forward and backward** easily.
The list also maintains a **head pointer**, which points to the first node.

✅ Advantages of Doubly Linked List

1. Bidirectional traversal

2. Easy deletion

3. Faster operations in the middle

✅ Algorithm to Insert a Node at the Beginning (DLL)

```
1. Create a new node new_node
2. Set new_node.data = value
3. Set new_node.prev = NULL
4. Set new_node.next = head

5. If head is NOT NULL
       head.prev = new_node

6. Set head = new_node

7. End
```

Q1 (B)Explain single link list with its advantage and write a algorithm to delete node at last from single like lost.

A **Singly Linked List** is a linear data structure made up of nodes.
Each node contains two parts:

1. **Data** – stores the value
2. **Next pointer** – stores the address of the next node in the sequence

The linked list starts with a **head pointer** that points to the first node.
The last node has its **next pointer = NULL**, showing the end of the list.

In a singly linked list, traversal is only **forward** (one direction).

✅ Advantages of Singly Linked List

1. Dynamic size

2. Easy insertion and deletion

3. Efficient memory usage

✅ Algorithm to Delete Last Node in a Singly Linked List

```
1. If head is NULL
       List is empty → return NULL

2. If head.next is NULL
       Only one node → delete it
       return NULL

3. Set temp = head

4. While temp.next.next != NULL
       Move temp = temp.next
       (temp will stop at second-last node)

5. Set temp.next = NULL
   (This removes the last node)

6. Return head

7. End
```

Q. 2(A) Define sorting show algorithm for marge sort with example

Sorting means arranging data in **ascending** or **descending** order.
Example:
`[5, 2, 8] → [2, 5, 8]`

✅ Merge Sort (Short Definition)

Merge Sort is a **divide-and-conquer** algorithm that:

1. **Divides** the list into two halves
2. **Sorts** each half
3. **Merges** them back in sorted order

✅ Algorithm (Easy)

```
Merge(left, right):

1. Create empty result list
2. Compare elements from left and right
3. Put smaller element into result
4. Continue until both lists are empty
5. Return result
```

✅ Example

Sort: `[8, 3, 5, 2]`

- Divide → `[8, 3]`, `[5, 2]`
- Divide again → `[8]` `[3]` and `[5]` `[2]`
- Merge → `[3, 8]` and `[2, 5]`
- Final merge → **[2, 3, 5, 8]**

Q. 2(B) Explain the working principle of quick sort and sort the following ist using quick sort [4,2,1,8,6,5,73,10,]

Here is the **short and easy** explanation of **Quick Sort** with the sorted list.

✅ Working Principle of Quick Sort (Easy)

Quick Sort works using **divide and conquer**:

1. **Choose a pivot** (a key element)
2. **Partition** the list:
    - elements **smaller** than pivot go to the left
    - elements **greater** than pivot go to the right
3. **Recursively** apply quick sort to left and right parts
4. Combine results → sorted list

Sorting [4,2,1,8,6,5,7,3,10]

Taking **4** as pivot:

Left: [2,1,3]
Right: [8,6,5,7,10]

Sort left → [1,2,3]
Sort right → [5,6,7,8,10]

Final:

[1,2,3,4,5,6,7,8,10]

Q. 2(C) Devlope a program for linked search and explain with example.

Explanation:

Linked search means checking each node one-by-one until the required value is found.

If the value matches → return "FOUND",

Otherwise go to next node.

"PROGRAM"

```
Def search(head, key):

    Temp = head
```

While temp is not None:

If temp.data == key: # match found

Return True

Temp = temp.next # move to next node

Return False # not found

✅ **Example**

Linked list: 10 → 20 → 30 → 40

Search for 30:

* Compare with 10 → no

* Compare with 20 → no

* Compare with 30 → yes → FOUND

Search for 50→ reaches end → NOT FOUND

Q. 3(A) Define level, degree, leaf node, sibling node with example.

1. Level

Level means the position of a node in a tree.

* **Root node → Level 0**

* Children of root → **Level 1**

* Their children → **Level 2**, and so on.

Example:

A (Level 0)

/ \

B C (Level 1)

/ \

D E (Level 2)

*2. Degree**

Degree of a node = Number of children that node has.

Example:

A has children B and C → Degree = 2

C has children D and E → Degree = 2

B has no children → Degree = 0

3. Leaf Node**

A **leaf node** is a node **with no children**.

Example:

In this tree:



Leaf nodes are: **B, D, E**

4. Sibling Nodes**

Nodes that have the **same parent** are called **siblings**.

Example:



Here **B and C** are **siblings** because their parent is **A**.

Q. 3(B) Snow the difference between Binary tree and Binary search.

Binary Tree	Binary Search Tree (BST)
A tree where **each node can have max 2 children**.	A special binary tree where **left < root < right** rule is followed.
No ordering rule between nodes.	**Ordered structure** for fast searching.
Used for general tree representation.	Used to **search, insert, delete** fast.
Example structure: children can be anywhere.	Example structure: left child is smaller, right is larger.